qsBye

rp2040笔记[2]-使用pio实现dshot并驱动电机

摘要

基于rust和embassy框架使用pio实现dshot并驱动电机;使用全局伪静态变量记录电机速度并异步更新值.
Implementing DShot and driving the motor using PIO based on Rust and embassy frameworks; Use global pseudo static variables to record motor speed and asynchronously update values.

关键词

rust;embassy;pio;dshot;mutex;

关键信息

- 项目地址:[https://github.com/ByeIO/byeefree.rp2040_quad.embassy]

```
[package]
edition = "2021"
name = "byeefree_rp2040_quad_embassy"
version = "0.1.0"
license = "MIT OR Apache-2.0"
resolver = "2"

[dependencies]
# arm cortex-m相关
cortex-m = { version = "0.7", features = ["inline-asm"] }
cortex-m-rt = "0.7"

# rp-pico抽象层相关
# rp-pico = "0.9.0"

# 调试信息相关
defmt = "0.3"
defmt-rtt = "0.4"
panic-probe = { version = "0.3", features = ["print-defmt"] }

# embassy嵌入式异步框架相关
embassy-embedded-hal = { version = "0.3", features = ["defmt"] }

embassy-executor = { version = "0.7", features = ["arch-cortex-m", "executor-thread", "execut
embassy-futures = { version = "0.1" }
embassy-sync = { version = "0.6", features = ["defmt"] }
embassy-time = { version = "0.4", features = ["defmt", "defmt-timestamp-uptime"] }

embassy-rp = { version = "0.3", features = ["defmt", "unstable-pac", "time-driver", "critical-

# pio相关, 暂时使用0.2版本
pio-proc = "0.2"
pio = "0.2"

## 通过usb获取日志信息 (注意: embassy-usb-logger 需要 portable-atomic 开启 critical-section特性)
embassy-usb-driver = { version = "0.1", features = ["defmt"] }
embassy-usb-logger = { version = "0.4" }
portable-atomic = { version = "1.10", features = ["critical-section"] }
log = "0.4"
embassy-usb = { version = "0.4.0", features = ["defmt"] }

## 网络支持
embassy-net = { version = "0.6", features = ["defmt", "tcp", "udp", "dhcpv4", "medium-etherne
# wiznet ethernet driver support
embassy-net-wiznet = { version = "0.2", features = ["defmt"] }
## pico-w 支持
cyw43 = { version = "0.3", features = ["defmt", "firmware-logs"] }
cyw43-pio = { version = "0.3", features = ["defmt"] }

# 生命周期
static_cell = "2.1.0"

# Dshot协议
dshot-pio = { path = "./static/dshot-pio", features = ["embassy-rp"] }

# 线性代数
nalgebra = { version = "0.33.2", default-features = false, features = ["libm"] }

# 浮点数计算支持
libm = "0.2.11"
num-traits = { version = "0.2", default-features = false, features = ["libm"] }

# 命令行解析
embedded-cli = "0.2.1"
embedded-io = "0.6.1"
embedded-io-async = "0.6.1"
ufmt = "0.2.0"

# 内存分配
embedded-alloc = { version = "0.6.0", features = ["llff"] }
heapless = { version = "0.8.0", features = ["portable-atomic"] }

# 临界区访问
critical-section = { version = "1.2.0", features = [] }

# cargo build/run
[profile.dev]
codegen-units = 1
debug = 2
debug-assertions = true
incremental = false
opt-level = 1
overflow-checks = true
lto = "off"

# cargo build/run --release
[profile.release]
codegen-units = 1
debug = 2
debug-assertions = false
incremental = false
lto = 'fat'
opt-level = 's'
overflow-checks = false

# do not optimize proc-macro crates = faster builds from scratch
[profile.dev.build-override]
codegen-units = 8
debug = false
debug-assertions = false
opt-level = 0
overflow-checks = false

[profile.release.build-override]
codegen-units = 8
debug = false
debug-assertions = false
opt-level = 0
overflow-checks = false

# cargo test
[profile.test]
codegen-units = 1
debug = 2
debug-assertions = true
incremental = false
opt-level = 's'
overflow-checks = true

# cargo test --release
[profile.bench]
codegen-units = 1
debug = 2
debug-assertions = false
incremental = false
lto = 'fat'
opt-level = 's'
```

原理简介

dshot/oneshot/multishot协议简介

[https://github.com/betaflight/betaflight/wiki/Dshot]
[https://www.bilibili.com/video/BV1jK411p7Yr/]
[https://team-blacksheep.freshdesk.com/support/solutions/articles/4000100133-what-are-oneshoot-multishot-and-dshot-]
[https://www.mathaelectronics.com/what-are-oneshot-and-multishot-in-esc-oneshot-vs-multishot-esc-esc-calibration-protocol/]
ESC校准协议：Oneshot与Multishot的核心解析

1. **ESC的定义与作用**
   电子调速器（ESC）是控制无刷直流电机（BLDC）转速、方向及制动的核心电路，其性能依赖于固件支持的通信协议。
2. **ESC固件类型**
   - 常见固件包括：BLHeli、BLHeli_S、SimonK、Kiss、BLHeli_32等。
   - 不同硬件支持的固件有限，例如BLHeli_S仅适用于特定ESC芯片。
3. **协议分类与特性**
   - **模拟PWM**：通过占空比控制电机（0%为停止，100%为全速），但延迟较高。
   - **标准PWM**：以脉冲宽度（1ms为停止，2ms为全速）传输指令，最大频率500Hz，延迟约2ms。
   - **Oneshot125**：
     - 脉冲范围缩短为125μs-250μs，频率提升至4000Hz，延迟降低至约250μs。
     - 兼容性广，适用于多数支持BLHeli固件的ESC。
   - **Oneshot42**：
     - 脉冲范围进一步压缩至42μs-84μs，频率更高，延迟更低，但对硬件要求更苛刻。
   - **Multishot**：
     - 目前最快的模拟协议，脉冲范围5μs-25μs，频率达32kHz，延迟仅约25μs，适用于高性能竞速无人机。
   - **Dshot（数字协议）**：
     - 通过数字信号传输（如Dshot150/300/600/1200），抗干扰能力强，无需校准，支持错误校验。
4. **Oneshot与Multishot的对比**

参数	Oneshot125	Multishot
脉冲范围	125-250μs	5-25μs
延迟	~250μs	~25μs
适用场景	通用飞行	高响应竞速
兼容性	广泛	需新硬件支持

5. **协议选择建议**
   - **优先数字协议**：Dshot因其抗噪性和稳定性逐渐成为主流。
   - **模拟协议场景**：若需极致低延迟（如竞速无人机），Multishot优于Oneshot；普通场景可选Oneshot125以兼容旧硬件。

dshot-pio库简介

[https://github.com/peterkrull/dshot-pio]
DShot implementation for RP2040 using PIO
This crate utilizes a single PIO block of the RP2040, enable it to send up to four DShot ESC commands (per PIO block, there are 2 of them) simultaneously, making it a great fit for quad-copters. The crate supports both the embassy-rp and rp2040-hal hardware abstraction layers (HAL).

rust no_std环境下全局变量小技巧

[https://blog.csdn.net/jxandrew/article/details/139348634]
[https://techie-s.work/posts/2023/01/embassy-an-easy-to-use-async-framework-for-embedded-rust/]
[https://blog.theembeddedrustacean.com/sharing-data-among-tasks-in-rust-embassy-synchronization-primitives#heading-the-async-mutex-type]

- 用静态变量互斥访问/写入来对冲rust的生命周期、借用难

1. 阻塞型
   在实例化共享全局变量时，需要定义RawMutex类型。在这里，我选择了ThreadModeMutex。选择什么取决于您使用互斥体的上下文。在我们的背景下，我们选择什么并不重要。更多细节可以在这里找到。
   在全局变量安装中，Mutex内的u32类型被RefCell封装。这是为了允许互斥体中包裹的值的内部可变性。要访问SHARED，使用lock方法获取全局共享值的锁。lock提供对值发生突变的闭包中锁定变量的访问。

```
use core::cell::RefCell;
use embassy_sync::blocking_mutex::Mutex;

static SHARED: Mutex<ThreadModeRawMutex, RefCell<u32>> = Mutex::new(RefCell::new(0));

#[embassy_executor::task]
async fn async_task() {
    loop {
        // Load value from global context, modify and store
        SHARED.lock(|f| {
            let val = f.borrow_mut().wrapping_add(1);
            f.replace(val);
        });
        Timer::after(Duration::from_millis(1000)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default());
    // Create empty String for message
    let mut msg: String<8> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task()).unwrap();

    loop {
        // Wait 1 second
        Timer::after(Duration::from_millis(1000)).await;
        // Obtain updated value from global context
        let shared = SHARED.lock(|f| f.clone().into_inner());
        core::writeln!(&mut msg, "{:02}", shared).unwrap();
        // Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}
```

2. 异步型
   那么，阻塞互斥体与异步互斥体有何不同？阻塞互斥锁不会在等待点/异步任务之间保持。或者，使用异步互斥体，可以在按住锁的同时等待，其他任务也会相应地等待。
   下面的代码在稍作修改后实现了与早期代码相同的功能。首先，请注意不再需要RefCell，并且共享变量不能通过闭包访问。相反，锁方法会等待锁被获取，并且在作用域结束之前不会释放它。如果你注意到async_task中锁的作用域，则有一个 `Timer::after(Duration::from_millis(1000)).await;` 插入的行。尝试一下此行中的延迟值，看看代码的行为会很有趣。关键是要证明，当你增加值时，锁将由async_task保持，防止主任务中的SHARED被访问。

```
use embassy_sync::blocking_mutex::raw::ThreadModeRawMutex;
use embassy_sync::mutex::Mutex;

static SHARED: Mutex<ThreadModeRawMutex, u32> = Mutex::new(0);

#[embassy_executor::task]
async fn async_task() {
    loop {
        {
            let mut shared = SHARED.lock().await;
            *shared = shared.wrapping_add(1);
            Timer::after(Duration::from_millis(1000)).await;
        }
        Timer::after(Duration::from_millis(1000)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    // Initialize and create handle for device peripherals
    let p = embassy_stm32::init(Default::default());
    //Configure UART
    let mut usart = UartTx::new(p.USART2, p.PA2, NoDma, Config::default());
    // Create empty String for message
    let mut msg: String<8> = String::new();
    // Spawn async blinking task
    spawner.spawn(async_task()).unwrap();

    loop {
        // Wait 1 second
        Timer::after(Duration::from_millis(1000)).await;
        // Obtain updated value from global context
        let shared = SHARED.lock().await;
        core::writeln!(&mut msg, "{:02}", *shared).unwrap();
        // Transmit Message
        usart.blocking_write(msg.as_bytes()).unwrap();
        msg.clear();
    }
}
```

关键代码

dshot_embassy_rp.rs

```
#![no_std]

#[cfg(feature = "embassy-rp")]
pub mod dshot_embassy_rp;

#[cfg(feature = "rp2040-hal")]
pub mod dshot_rp2040_hal;

pub trait DshotPioTrait<const N: usize> {
    fn command(&mut self, command: [u16;N]);
    fn reverse(&mut self, reverse: [bool;N]);
    fn throttle_clamp(&mut self, throttle: [u16;N]);
    fn throttle_minimum(&mut self);
}

// 各种Shot类型
pub enum ShotType{
    DSHOT150,
    DSHOT300,
    DSHOT600,
    DSHOT1200,
    OneSHOT42,
    OneShot125,
    MultiShot
}

// 计算分频数
pub fn cal_clock_div(sys_clk_hz: u32, shot_type: ShotType) -> (u16, u8) {
    let bit_rate = match shot_type {
        ShotType::DSHOT150 => 150_000,
        ShotType::DSHOT300 => 300_000,
        ShotType::DSHOT600 => 600_000,
        ShotType::DSHOT1200 => 1_200_000,
        // 默认DShot600, 或根据需要panic
        _ => 600_000,
    };

    let denominator = 8u64 * bit_rate as u64;
    let sys_clk = sys_clk_hz as u64;

    let divider = (sys_clk / denominator) as u16;
    let remainder = sys_clk % denominator;
    // (remainder * 256)/denominator
    let fraction = ((remainder << 8) / denominator) as u8;

    (divider, fraction)
}
use dshot_encoder as dshot;
pub use super::DshotPioTrait;

pub use embassy_rp::{
    pio::{ Instance, Pio, Config, PioPin, ShiftConfig, ShiftDirection::Left, InterruptHandler
    Peripheral, interrupt::typelevel::Binding
};
#[allow(dead_code)]
pub struct DshotPio<'a, const N : usize, PIO : Instance> {
    pio_instance: Pio<'a,PIO>,
}


fn configure_pio_instance<'a,PIO: Instance>  (
    pio: impl Peripheral<P = PIO> + 'a,
    irq: impl Binding<PIO::Interrupt, InterruptHandler<PIO>>,
    clk_div: (u16, u8),
) -> (Config<'a, PIO>, Pio<'a, PIO>) {

    // Define program
    let dshot_pio_program = pio_proc::pio_asm!(
        "set pindirs, 1",
        "entry:"
        "   pull"
        "   out null 16"
        "   set x 15"
        "loop:"
        "   set pins 1"
        "   out y 1"
        "   jmp !y zero"
        "   nop [2]"
        "one:" // 6 and 2
        "   set pins 0"
        "   jmp x-- loop"
        "   jmp reset"
        "zero:" // 3 and 5
        "   set pins 0 [3]"
        "   jmp x-- loop"
        "   jmp reset"
        "reset:" // Blank frame
        "   nop [31]"
        "   nop [31]"
        "   nop [31]"
        "   jmp entry [31]"
    );

    // Configure program
    let mut cfg = Config::default();
    let mut pio = Pio::new(pio,irq);
    cfg.use_program(&pio.common.load_program(&dshot_pio_program.program), &[]);
    cfg.clock_divider = clk_div.0.into();

    cfg.shift_in = ShiftConfig {
        auto_fill: true,
        direction: Default::default(),
        threshold: 32,
    };

    cfg.shift_out = ShiftConfig {
        auto_fill: Default::default(),
        direction: Left,
        threshold: Default::default(),
    };

    (cfg,pio)

}

///
/// Defining constructor functions
///

impl <'a,PIO: Instance> DshotPio<'a,1,PIO> {
    pub fn new(
        pio: impl Peripheral<P = PIO> + 'a,
        irq: impl Binding<PIO::Interrupt, InterruptHandler<PIO>>,
        pin0: impl PioPin,
        clk_div: (u16, u8),
    ) -> DshotPio<'a,1,PIO> {

        let (mut cfg, mut pio) = configure_pio_instance(pio, irq, clk_div);

        // Set pins and enable all state machines
        let pin0 = pio.common.make_pio_pin(pin0);
        cfg.set_set_pins(&[&pin0]);
        pio.sm0.set_config(&cfg);
        pio.sm0.set_enable(true);

        // Return struct of 1 configured DShot state machine
        DshotPio { pio_instance : pio }
    }
}

impl <'a,PIO: Instance> DshotPio<'a,2,PIO> {
    pub fn new(
        pio: impl Peripheral<P = PIO> + 'a,
        irq: impl Binding<PIO::Interrupt, InterruptHandler<PIO>>,
        pin0: impl PioPin,
        pin1: impl PioPin,
        clk_div: (u16, u8),
    ) -> DshotPio<'a,2,PIO> {

        let (mut cfg, mut pio) = configure_pio_instance(pio, irq, clk_div);

        // Set pins and enable all state machines
        let pin0 = pio.common.make_pio_pin(pin0);
        cfg.set_set_pins(&[&pin0]);
        pio.sm0.set_config(&cfg);
        pio.sm0.set_enable(true);

        let pin1 = pio.common.make_pio_pin(pin1);
        cfg.set_set_pins(&[&pin1]);
        pio.sm1.set_config(&cfg);
        pio.sm1.set_enable(true);

        // Return struct of 2 configured DShot state machines
        DshotPio { pio_instance : pio }
    }
}

impl <'a,PIO: Instance> DshotPio<'a,3,PIO> {
    pub fn new(
        pio: impl Peripheral<P = PIO> + 'a,
        irq: impl Binding<PIO::Interrupt, InterruptHandler<PIO>>,
        pin0: impl PioPin,
        pin1: impl PioPin,
        pin2: impl PioPin,
        clk_div: (u16, u8),
    ) -> DshotPio<'a,3,PIO> {

        let (mut cfg, mut pio) = configure_pio_instance(pio, irq, clk_div);

        // Set pins and enable all state machines
        let pin0 = pio.common.make_pio_pin(pin0);
        cfg.set_set_pins(&[&pin0]);
        pio.sm0.set_config(&cfg);
        pio.sm0.set_enable(true);

        let pin1 = pio.common.make_pio_pin(pin1);
        cfg.set_set_pins(&[&pin1]);
        pio.sm1.set_config(&cfg);
        pio.sm1.set_enable(true);

        let pin2 = pio.common.make_pio_pin(pin2);
        cfg.set_set_pins(&[&pin2]);
        pio.sm2.set_config(&cfg);
        pio.sm2.set_enable(true);

        // Return struct of 3 configured DShot state machines
        DshotPio { pio_instance : pio }
    }
}

impl <'a,PIO: Instance> DshotPio<'a,4,PIO> {
    pub fn new(
        pio: impl Peripheral<P = PIO> + 'a,
        irq: impl Binding<PIO::Interrupt, InterruptHandler<PIO>>,
        pin0: impl PioPin,
        pin1: impl PioPin,
        pin2: impl PioPin,
        pin3: impl PioPin,
        clk_div: (u16, u8),
    ) -> DshotPio<'a,4,PIO> {

        let (mut cfg, mut pio) = configure_pio_instance(pio, irq, clk_div);

        // Set pins and enable all state machines
        let pin0 = pio.common.make_pio_pin(pin0);
        cfg.set_set_pins(&[&pin0]);
        pio.sm0.set_config(&cfg);
        pio.sm0.set_enable(true);

        let pin1 = pio.common.make_pio_pin(pin1);
        cfg.set_set_pins(&[&pin1]);
        pio.sm1.set_config(&cfg);
        pio.sm1.set_enable(true);

        let pin2 = pio.common.make_pio_pin(pin2);
        cfg.set_set_pins(&[&pin2]);
        pio.sm2.set_config(&cfg);
        pio.sm2.set_enable(true);

        let pin3 = pio.common.make_pio_pin(pin3);
        cfg.set_set_pins(&[&pin3]);
        pio.sm3.set_config(&cfg);
        pio.sm3.set_enable(true);

        // Return struct of 4 configured DShot state machines
        DshotPio { pio_instance : pio }
    }
}

///
/// Implementing DshotPioTrait
///

impl <'d,PIO : Instance> super::DshotPioTrait<1> for DshotPio<'d,1,PIO> {

    /// Send any valid DShot value to the ESC.
    fn command(&mut self, command: [u16; 1]) {
        self.pio_instance.sm0.tx().push(command[0].min(dshot::THROTTLE_MAX) as u32);
    }

    /// Set the direction of rotation for each motor
    fn reverse(&mut self, reverse: [bool;1]) {
        self.pio_instance.sm0.tx().push(dshot::reverse(reverse[0]) as u32);
    }

    /// Set the throttle for each motor. All values are clamped between 48 and 2047
    fn throttle_clamp(&mut self, throttle: [u16;1]) {
        self.pio_instance.sm0.tx().push(dshot::throttle_clamp(throttle[0], false) as u32);
    }

    /// Set the throttle for each motor to zero (DShot command 48)
    fn throttle_minimum(&mut self) {
        self.pio_instance.sm0.tx().push(dshot::throttle_minimum(false) as u32);
    }
}

impl <'d,PIO : Instance> super::DshotPioTrait<2> for DshotPio<'d,2,PIO> {

    /// Send any valid DShot value to the ESC.
    fn command(&mut self, command: [u16; 2]) {
        self.pio_instance.sm0.tx().push(command[0].min(dshot::THROTTLE_MAX) as u32);
        self.pio_instance.sm0.tx().push(command[1].min(dshot::THROTTLE_MAX) as u32);
    }

    /// Set the direction of rotation for each motor
    fn reverse(&mut self, reverse: [bool;2]) {
        self.pio_instance.sm0.tx().push(dshot::reverse(reverse[0]) as u32);
        self.pio_instance.sm1.tx().push(dshot::reverse(reverse[1]) as u32);
    }

    /// Set the throttle for each motor. All values are clamped between 48 and 2047
    fn throttle_clamp(&mut self, throttle: [u16;2]) {
        self.pio_instance.sm0.tx().push(dshot::throttle_clamp(throttle[0], false) as u32);
        self.pio_instance.sm1.tx().push(dshot::throttle_clamp(throttle[1], false) as u32);
    }

    /// Set the throttle for each motor to zero (DShot command 48)
    fn throttle_minimum(&mut self) {
        self.pio_instance.sm0.tx().push(dshot::throttle_minimum(false) as u32);
        self.pio_instance.sm1.tx().push(dshot::throttle_minimum(false) as u32);
    }
}

impl <'d,PIO : Instance> super::DshotPioTrait<3> for DshotPio<'d,3,PIO> {

    /// Send any valid DShot value to the ESC.
    fn command(&mut self, command: [u16; 3]) {
        self.pio_instance.sm0.tx().push(command[0].min(dshot::THROTTLE_MAX) as u32);
        self.pio_instance.sm0.tx().push(command[1].min(dshot::THROTTLE_MAX) as u32);
        self.pio_instance.sm0.tx().push(command[2].min(dshot::THROTTLE_MAX) as u32);
    }

    /// Set the direction of rotation for each motor
    fn reverse(&mut self, reverse: [bool;3]) {
        self.pio_instance.sm0.tx().push(dshot::reverse(reverse[0]) as u32);
        self.pio_instance.sm1.tx().push(dshot::reverse(reverse[1]) as u32);
        self.pio_instance.sm2.tx().push(dshot::reverse(reverse[2]) as u32);
    }

    /// Set the throttle for each motor. All values are clamped between 48 and 2047
    fn throttle_clamp(&mut self, throttle: [u16;3]) {
        self.pio_instance.sm0.tx().push(dshot::throttle_clamp(throttle[0], false) as u32);
        self.pio_instance.sm1.tx().push(dshot::throttle_clamp(throttle[1], false) as u32);
        self.pio_instance.sm2.tx().push(dshot::throttle_clamp(throttle[2], false) as u32);
    }

    /// Set the throttle for each motor to zero (DShot command 48)
    fn throttle_minimum(&mut self) {
        self.pio_instance.sm0.tx().push(dshot::throttle_minimum(false) as u32);
        self.pio_instance.sm1.tx().push(dshot::throttle_minimum(false) as u32);
        self.pio_instance.sm2.tx().push(dshot::throttle_minimum(false) as u32);
    }
}

impl <'d,PIO : Instance> super::DshotPioTrait<4 > for DshotPio<'d,4,PIO> {

    /// Send any valid DShot value to the ESC.
    fn command(&mut self, command: [u16; 4]) {
        self.pio_instance.sm0.tx().push(command[0].min(dshot::THROTTLE_MAX) as u32);
        self.pio_instance.sm0.tx().push(command[1].min(dshot::THROTTLE_MAX) as u32);
        self.pio_instance.sm0.tx().push(command[2].min(dshot::THROTTLE_MAX) as u32);
        self.pio_instance.sm0.tx().push(command[3].min(dshot::THROTTLE_MAX) as u32);
    }

    /// Set the direction of rotation for each motor
    fn reverse(&mut self, reverse: [bool;4]) {
        self.pio_instance.sm0.tx().push(dshot::reverse(reverse[0]) as u32);
        self.pio_instance.sm1.tx().push(dshot::reverse(reverse[1]) as u32);
        self.pio_instance.sm2.tx().push(dshot::reverse(reverse[2]) as u32);
        self.pio_instance.sm3.tx().push(dshot::reverse(reverse[3]) as u32);
    }

    /// Set the throttle for each motor. All values are clamped between 48 and 2047
    fn throttle_clamp(&mut self, throttle: [u16;4]) {
        self.pio_instance.sm0.tx().push(dshot::throttle_clamp(throttle[0], false) as u32);
        self.pio_instance.sm1.tx().push(dshot::throttle_clamp(throttle[1], false) as u32);
        self.pio_instance.sm2.tx().push(dshot::throttle_clamp(throttle[2], false) as u32);
        self.pio_instance.sm3.tx().push(dshot::throttle_clamp(throttle[3], false) as u32);
    }

    /// Set the throttle for each motor to zero (DShot command 48)
    fn throttle_minimum(&mut self) {
        self.pio_instance.sm0.tx().push(dshot::throttle_minimum(false) as u32);
        self.pio_instance.sm1.tx().push(dshot::throttle_minimum(false) as u32);
        self.pio_instance.sm2.tx().push(dshot::throttle_minimum(false) as u32);
        self.pio_instance.sm3.tx().push(dshot::throttle_minimum(false) as u32);
    }
}
```

motors.rs

```
#![allow(unused)]

//! 电机控制任务
// rp2040-pico默认时钟频率: 120MHz
// 小蜜蜂电调,BLHeli-S固件, DShot600控制

// 多任务相关
use embassy_time::{Duration, Timer};
use embassy_executor::Spawner;
use embassy_futures::join::join;
use embassy_rp::bind_interrupts;

// 电调驱动
use crate::{drivers::esc::little_bee, utils::signals::MOTOR_STATE};

// dshot相关
use dshot_pio::{ cal_clock_div, dshot_embassy_rp::DshotPio, ShotType, DshotPioTrait };

// 静态变量互斥访问相关
use embassy_sync::blocking_mutex::{Mutex, CriticalSectionMutex, raw::CriticalSectionRawMutex}
// use core::sync::atomic::{AtomicU16, Ordering};
use static_cell::StaticCell;
use core::{
    //内部可变性类型RefCell
    cell::RefCell,
    fmt::Write,
    //引用借用
    borrow::BorrowMut,
    borrow::Borrow,
};

// 内部库
/// 信号
use crate::utils::signals;
/// 全局变量
use crate::utils::variables;

// 电机序号
#[derive(Clone,Copy,PartialEq)]
pub enum Motors {
    One,
    Two,
    Three,
    Four
}

// 电机状态
#[derive(Clone,Copy,PartialEq)]
pub enum MotorState {
    READY,
    DISARMED,
    KEEP,
    RUNNING,
    MANUAL,
}

// 1. 解锁电调工具函数
pub async fn arming_esc(
    motor: Motors,
    out_pio_motors : &mut DshotPio<'static, 4, embassy_rp::peripherals::PIO0>
    ){
    defmt::println!("hello from arming_esc");
    for _i in 0..50 {
        out_pio_motors.throttle_minimum();
        Timer::after(Duration::from_millis(50)).await;
    }
}

// 2. 电机转速控制任务(async)
// motor序号(e.g. Motors::One), 0~100转速(u16)
#[embassy_executor::task]
pub async fn motor_task(
    mut in_motor_speed : signals::MotorSpeedSub,
    mut in_motor_direction : signals::MotorDirSub,
    mut out_pio_motors : DshotPio<'static, 4, embassy_rp::peripherals::PIO0>,
    out_motor_state : signals::MotorStatePub,
){
    defmt::println!("hello from motor_task");

    // 初始化电机
    init_motors(&mut out_pio_motors).await;
```

公告

昵称： qsBye
园龄： 8年4个月
粉丝： 27
关注： 0

日	一	二	三	四	五	六
23	24	25	26	27	28	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31	1	2	3	4	5

‹ 2025年3月 ›

搜索

常用链接

- 我的随笔
- 我的评论
- 我的参与
- 最新评论
- 我的标签

我的标签

- Rust(12)
- SLAM(5)
- 线性代数(2)
- LLM(1)

随笔分类

- 51单片机(10)
- Android开发(18)
- bouffalo单片机(1)
- esp32单片机(24)
- GNU Radio(4)
- LLM精彩问答(7)
- MATLAB(6)
- RaspberryPi单片机(6)
- SCILAB(3)
- stm32单片机(16)
- TI单片机(3)
- WebAssembly(3)
- 翻译(5)
- 高云FPGA(3)
- 公众号(2)
- 鸿蒙开发(3)
- 机器人(11)
- 前端开发(12)
- 嵌入式Linux(3)
- 人工智能(4)
- 运维相关(16)
- 折腾笔记(18)

随笔档案

- 2025年3月(3)
- 2025年2月(6)
- 2025年1月(11)
- 2024年12月(1)
- 2024年10月(1)
- 2024年9月(4)
- 2024年8月(2)
- 2024年6月(3)
- 2024年5月(9)
- 2024年4月(3)
- 2024年3月(10)
- 2024年2月(8)
- 2024年1月(9)
- 2023年12月(4)
- 2023年11月(6)
- 2023年10月(4)
- 2023年9月(4)
- 2023年8月(10)
- 2023年7月(16)
- 2023年6月(2)
- 2023年5月(2)
- 2023年4月(3)
- 2023年3月(2)
- 2023年2月(7)
- 2023年1月(17)
- 2022年12月(11)
- 2022年11月(14)
- 2022年10月(8)
- 2022年9月(17)
- 2022年8月(23)
- 更多

阅读排行榜

1. OpenWrt安装Filebrowser网页文件管理器(11434)
2. esp32笔记[5]-基于I2S协议实现音频播放(9262)
3. 谈谈NAS的硬盘SATA背板(6554)
4. 三星平板的笔记模板(比例:16:10)(4271)
5. 电犀牛R66s软路由体验(4227)

评论排行榜

1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜

1. esp32笔记[5]-基于I2S协议实现音频播放(1)
2. esp32笔记[1]-blink闪灯(1)
3. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
4. 基于esp32的MQTT开关门检测模块(1)
5. 51单片机笔记[6]-电机控制(1)

最新评论

1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--赤Xin
2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢?
我按照本文给出的方案安装之后docker容量只有4G
--VantHoff
3. Re:51单片机笔记[1]-基础实验
大学买的单片机就没打开过.
--mossxzzom

```
    out_motor_state.publish_immediate(MotorState::READY);

    defmt::println!("done: motor init");

    loop{
        // 1. 根据转速信号调整电机
        /// 获取电机转速
        if let Some(motor_speed) = in_motor_speed.try_next_message_pure(){
            let (m1, m2, m3, m4) = motor_speed;

            /// 更新全局变量
            crate::utils::variables::MotorSpeedEditor::write_speeds(
                [m1.unwrap_or(0),m2.unwrap_or(0),m3.unwrap_or(0),m4.unwrap_or(0)]
            ).await;

            /// 发送到电调
            out_pio_motors.throttle_clamp([
                m1.unwrap_or(0),
                m2.unwrap_or(0),
                m3.unwrap_or(0),
                m4.unwrap_or(0)
            ]);
        }

        // 2. 根据方向给电机换向
        /// 获取电机旋转方向设置
        // let mut motor_dir = in_motor_direction.next_message_pure().await;
        if let Some(motor_dir) = in_motor_direction.try_next_message_pure(){
            let (m1_, m2_, m3_, m4_) = motor_dir;
            /// 更新全局变量
            crate::utils::variables::MotorDirectionEditor::write_directions(
                [m1_.unwrap_or(true),m2_.unwrap_or(true),m3_.unwrap_or(true),m4_.unwrap_or(tr
            ).await;

            /// 发送到电调
            out_pio_motors.reverse([
                m1_.unwrap_or(true),
                m2_.unwrap_or(true),
                m3_.unwrap_or(true),
                m4_.unwrap_or(true)
            ]);
        }else{
            // 留出时间给其他任务执行，不能一直占用cpu
            Timer::after(Duration::from_millis(50)).await;
        }

    }
}

// 3. 电调初始化
// p_pio, (p_motor1_gpio, p_motor2_gpio, p_motor3_gpio, p_motor4_gpio)
pub async fn init_motors(out_pio_motors :&mut DshotPio<'static, 4, embassy_rp::peripherals::P
{
    defmt::println!("hello from init_motors");
    // 3.1 绑定gpio和dshot信号
    let _ = little_bee::init().await;

    // 3.2 发送小油门信号解锁电调，让电机可以旋转
    arming_esc(Motors::One, out_pio_motors).await;
    arming_esc(Motors::Two, out_pio_motors).await;
    arming_esc(Motors::Three, out_pio_motors).await;
    arming_esc(Motors::Four, out_pio_motors).await;
}

// 4. 测试分别旋转四个电机，然后让电机保持缓慢旋转
#[embassy_executor::task]
pub async fn motor_test_task(
    mut in_motor_state : signals::MotorStateSub,
){
    // 等待电机初始化完成
    let state = in_motor_state.next_message_pure().await;
    if state == MotorState::READY {
        defmt::println!("hello from motor_test");

        // 1. 依次旋转四个电机
        signals::MOTOR_SPEED.publisher().unwrap().publish_immediate((Some(30), Some(0), Some(
        Timer::after(Duration::from_millis(2000)).await;
        signals::MOTOR_SPEED.publisher().unwrap().publish_immediate((Some(0), Some(30), Some(
        Timer::after(Duration::from_millis(2000)).await;
        signals::MOTOR_SPEED.publisher().unwrap().publish_immediate((Some(0), Some(0), Some(3
        Timer::after(Duration::from_millis(2000)).await;
        signals::MOTOR_SPEED.publisher().unwrap().publish_immediate((Some(0), Some(0), Some(0
        Timer::after(Duration::from_millis(2000)).await;

        defmt::println!("done: motor_test");

        // 2. 保持慢速旋转，需要一直发布信号
        loop{
            if let Some(state2) = in_motor_state.try_next_message_pure(){
                if state2 == MotorState::READY {
                    signals::MOTOR_SPEED.publisher().unwrap().publish_immediate((Some(10), So
                    // 留出时间给其他任务执行，不能一直占用cpu
                    Timer::after(Duration::from_millis(500)).await;
                }else{
                    defmt::println!("done: motor_ready");
                    // 退出loop循环
                    break;
                }// end else
            }// end if let Some
            else{
                signals::MOTOR_SPEED.publisher().unwrap().publish_immediate((Some(10), Some(1
                // 留出时间给其他任务执行，不能一直占用cpu
                Timer::after(Duration::from_millis(500)).await;
            }// end else

        }// end loop

    }
}
```

motor.rs

```
// 命令解析相关
use embedded_cli::Command;
use embedded_io::ErrorKind;
use embedded_io_async::{Read, Write};
use ufmt::uwrite;
use heapless::String;
use core::str::FromStr;
use crate::shell::{UBuffer, INTERRUPT};

// 异步相关
use embassy_time::{Duration, Ticker};
use embassy_futures::select::{select, Either};

// 内部库
/// 信号
use crate::utils::signals;
/// 全局变量
use crate::utils::variables;

#[derive(Command, Clone)]
pub enum MotorCommand {
    /// set speed and direction
    Set {
        #[command(subcommand)]
        cmd: MotorSetEnum,
    },
    /// get speed and direction
    Get,
}

// 实现CommandHandler特征
impl super::CommandHandler for MotorCommand {
    async fn handler(
        &self,
        mut serial: impl Read<Error = ErrorKind> + Write<Error = ErrorKind>,
    ) -> Result<(), ErrorKind> {
        match self {
            // 具体解析命令
            MotorCommand::Set { cmd } => {
                cmd.handler(&mut serial).await?
            },
            MotorCommand::Get => {
                let speeds = crate::utils::variables::MotorSpeedEditor::get_speeds().await;
                let directions = crate::utils::variables::MotorDirectionEditor::get_direction
                // 使用UBuffer结构体
                let mut buf = UBuffer::<32>::new();

                // 遍历数组元素逐个写入
                for (i, speed) in speeds.as_ref().iter().enumerate() {
                    if i > 0 {
                        // 添加分隔符
                        uwrite!(&mut buf, " ")?;
                    }
                    uwrite!(&mut buf, "{}", speed)?;
                }

                // 加一个空格
                uwrite!(&mut buf, " ")?;

                // 调试信息打印
                defmt::println!("motor speed: {}", core::str::from_utf8(&buf.inner).unwrap())
                // 写入到输出接口
                serial.write_all(&buf.inner).await?;

                // 使用UBuffer结构体
                let mut buf2 = UBuffer::<32>::new();

                // 遍历数组元素逐个写入
                for (i, direction) in directions.as_ref().iter().enumerate() {
                    if i > 0 {
                        // 添加分隔符
                        uwrite!(&mut buf2, " ")?;
                    }
                    uwrite!(&mut buf2, "{}", direction)?;
                }

                // 调试信息打印
                defmt::println!("motor directions: {}", core::str::from_utf8(&buf2.inner).unw
                // 写入到输出接口
                serial.write_all(&buf2.inner).await?;
            }
        }
        Ok(())
    }
}

#[derive(Command, Clone)]
pub enum MotorSetEnum{
    // 设置速度，`motor set speed 1 100`
    /// set speed
    Speed{
        motor: usize,
        speed: u16,
    },
    // 设置方向，`motor set dir 1 false`
    /// set direction
    Dir{
        motor: usize,
        direction: bool,
    }
}

// 实现CommandHandler特征
impl super::CommandHandler for MotorSetEnum {
    async fn handler(
        &self,
        mut serial: impl Read<Error = ErrorKind> + Write<Error = ErrorKind>,
    ) -> Result<(), ErrorKind> {
        match self {
            MotorSetEnum::Speed{motor, speed}=>{
                // 转换电机模式到MANUAL
                signals::MOTOR_STATE.publisher().unwrap().publish_immediate(crate::tasks::mot

                // TODO 检查数值合法性

                // 获取全局变量的电机转速
                let mut speeds = crate::utils::variables::MotorSpeedEditor::get_speeds().awai
                speeds[motor-1] = *speed;

                // 发布电机转速设置
                signals::MOTOR_SPEED.publisher().unwrap().publish_immediate(
                    (Some(speeds[0]),Some(speeds[1]),Some(speeds[2]),Some(speeds[3]))
                );

                defmt::println!("speeds will be set to :{}", speeds);

                serial.write_all(b"ok").await?;
            },
            MotorSetEnum::Dir{motor, direction}=>{

                // TODO 检查数值合法性

                // 获取全局变量的电机方向
                let mut directions = crate::utils::variables::MotorDirectionEditor::get_direc
                directions[motor-1] = *direction;

                // 发布电机方向设置
                signals::MOTOR_DIR.publisher().unwrap().publish_immediate(
                    (Some(directions[0]),Some(directions[1]),Some(directions[2]),Some(directi
                );

                defmt::println!("direction will be set to :{}", directions);

                serial.write_all(b"ok").await?;
            }
        }
        Ok(())
    }
}
```

signals.rs

```
// 2. motors任务的信道
/// 电机转速
const MOTOR_SPEED_NUM: usize = 2;
/// 可对电机进行单独设置而不影响其他电机
pub type MotorSpeedType = (Option<u16>, Option<u16>, Option<u16>, Option<u16>);
pub type MotorSpeedPub = Pub<MotorSpeedType,MOTOR_SPEED_NUM>;
pub type MotorSpeedSub = Sub<MotorSpeedType,MOTOR_SPEED_NUM>;
pub static MOTOR_SPEED : Ch<MotorSpeedType,MOTOR_SPEED_NUM> = PubSubChannel::new();

/// 电机方向
const MOTOR_DIR_NUM: usize = 2;
/// 可对电机进行单独设置而不影响其他电机
pub type MotorDirType = (Option<bool>, Option<bool>, Option<bool>, Option<bool>);
pub type MotorDirPub = Pub<MotorDirType,MOTOR_DIR_NUM>;
pub type MotorDirSub = Sub<MotorDirType,MOTOR_DIR_NUM>;
pub static MOTOR_DIR : Ch<MotorDirType,MOTOR_DIR_NUM> = PubSubChannel::new();

/// 电机状态
const MOTOR_STATE_NUM: usize = 2;
pub type MotorStateType = crate::tasks::motors::MotorState;
pub type MotorStatePub = Pub<MotorStateType,MOTOR_STATE_NUM>;
pub type MotorStateSub = Sub<MotorStateType,MOTOR_STATE_NUM>;
pub static MOTOR_STATE : Ch<MotorStateType,MOTOR_STATE_NUM> = PubSubChannel::new();
```

variables.rs

```
#![allow(dead_code)]

//! 全局变量

// 用静态变量互斥访问来对冲rust的生命周期、借用难，
// 所有的操作接口都是：&'static self。

// 静态变量互斥访问相关
use embassy_sync::blocking_mutex::{
    CriticalSectionMutex,
    raw::{
        CriticalSectionRawMutex, ThreadModeRawMutex
    },
};
use embassy_sync::mutex::Mutex;
// use core::sync::atomic::{AtomicU16, Ordering};
use static_cell::{StaticCell, ConstStaticCell};
use core::{
    //内部可变性类型RefCell
    cell::RefCell,
    fmt::Write,
    //引用借用
    borrow::BorrowMut,
    borrow::Borrow,
};

/// 初始化全局变量
pub fn init_variables(){
    // unimplemented!()
}

/* start 电机速度 */
/// 全局电机速度，使用异步Mutex保护
static MOTOR_SPEEDS: Mutex<ThreadModeRawMutex, [u16; 4]> = Mutex::new([0u16; 4]);

/// 电机速度操作接口
pub struct MotorSpeedEditor;

impl MotorSpeedEditor {
    /// 异步获取电机速度（拷贝值返回）
    pub async fn get_speeds() -> [u16; 4] {
        let guard = MOTOR_SPEEDS.lock().await;
        *guard
    }

    /// 异步更新全部电机速度
    pub async fn write_speeds(speeds: [u16; 4]) {
        let mut guard = MOTOR_SPEEDS.lock().await;
        *guard = speeds;
    }

    /// 异步设置单个电机速度
    pub async fn set_speed(index: usize, value: u16) {
        let mut guard = MOTOR_SPEEDS.lock().await;
        guard[index] = value;
    }
}
/* end 电机速度 */

/* start 电机方向 */
// 1: 右转，0: 左转
/// 全局电机方向，使用异步Mutex保护
static MOTOR_DIRECTIONS: Mutex<ThreadModeRawMutex, [bool; 4]> = Mutex::new([true; 4]);

/// 电机方向操作接口
pub struct MotorDirectionEditor;

impl MotorDirectionEditor {
    /// 异步获取电机方向（拷贝值返回）
    pub async fn get_directions() -> [bool; 4] {
        let guard = MOTOR_DIRECTIONS.lock().await;
        *guard
    }

    /// 异步更新全部电机方向
    pub async fn write_directions(directions: [bool; 4]) {
        let mut guard = MOTOR_DIRECTIONS.lock().await;
        *guard = directions;
    }

    /// 异步设置单个电机方向
    pub async fn set_direction(index: usize, value: bool) {
        let mut guard = MOTOR_DIRECTIONS.lock().await;
        guard[index] = value;
    }
}
/* end 电机方向 */
```

分类: RaspberryPi单片机

标签: Rust

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐　0 反对

« 上一篇： 翻译[5]-基于rust和embassy异步框架的飞控固件: Holsatus Flight

posted @ 2025-03-20 10:40 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】抖音旗下AI助手豆包，你的智能百科全书，全免费不限次数
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

编辑推荐：
· AI与.NET技术实操系列：使用Catalyst进行自然语言处理
· 分享一个我遇到过的“量子力学”级别的BUG。
· Linux系列：如何调试 malloc 的底层源码
· AI与.NET技术实操系列：基于图像分类模型对图像进行分类
· go语言实现终端里的倒计时

阅读排行：
· 历时 8 年，我冲上开源榜前 8 了！
· 物流快递公司核心技术能力-海量大数据处理技术
· 四大AI编程工具组合测评
· 关于能否用DeepSeek做危险的事情，DeepSeek本身给出了答案
· 几个技巧，教你去除文章的 AI 味！

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes